BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g.,

`{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent`.
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*
   Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`
3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`

5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

1. Mathematical Mapping of Table-Driven Agents

An abstract Agent Function is mathematically defined as:
f : P* → A
where P* represents the set of all possible percept sequences (history of percepts) and A is the set of possible actions.
A Table-Driven Agent attempts to store this function directly as a massive lookup table containing pre-computed optimal actions for every possible sequence of percepts.

2. Combinatorial Explosion

If an environment has |P| possible distinct percepts per time step and operates over a lifetime of T steps, the total number of entries in the lookup table is given by:
Table Size = |P|^T
As the agent's lifetime T increases, the table size grows exponentially (Combinatorial Explosion).

3. The Chess Example

- Chess has roughly 10^40 legal board states.
- The total number of possible game move sequences is estimated at 10^120 (The Shannon Number).

4. Conclusion

Storing a lookup table for Chess would require more memory than there are atoms in the observable universe. No physical storage device can hold it, no human could write it, and no compiler could build it. Therefore, AI systems must use Agent Architectures that compute actions algorithmically on the fly rather than memorizing lookup tables.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

1. Food Detection(if percept["food_here"]: return "Stay"):

Condition: percept["food_here"] == True
Action: "Stay" (consume food).

2. Obstacle Avoidance (elif percept["wall_ahead"]: return random.choice(["Left", "Right"])):

Condition: percept["wall_ahead"] == True
Action: "Left" or "Right" (obstacle avoidance).

3. Clear Path (else: return "Down"):

Condition: Default / clear path ahead.
Action: "Down" (advance forward).

4. Theoretical Alignment

A Simple Reflex Agent operates strictly on:
state = INTERPRET-INPUT(percept) → rule = RULE-MATCH(state, rules) → action = rule.ACTION
It has zero internal memory (__init__ holds no history) and selects actions based only on the immediate current percept.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

1. Partial Observability

get_percept() was restricted to return only local boolean readings (wall_ahead and food_here), hiding global coordinates (x, y) and grid layout memory. The agent is blind to where it is in the world.

2. Lack of Percept History (Zero Memory)

The SimpleReflexAgent evaluates sense_and_act(self, percept) without recording previous steps or visited cells. At time step t, it has no awareness of where it was at time step t−1.

3. Mechanism of Failure (Infinite Loop)

- When the agent enters a corner or U-shaped obstacle wall, its sensors report wall_ahead: True.
- The reflex rule triggers an evasion turn (e.g., Left).
- On the following step, the reflex rule moves the agent into an adjacent cell where it perceives wall_ahead: False. It moves forward, hits the wall again, and receives the exact same local percept dictionary {'wall_ahead': True, 'food_here': False}.
- Because the agent lacks percept history, it cannot recognize that it has returned to an identical state. The deterministic/limited IF-THEN rules cause it to repeat the identical sequence of moves indefinitely, getting trapped in an infinite loop.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

1. Transition Model (How the world evolves / How actions change location)

Code Implementation: Updates relative internal coordinates (x, y) based on self.last_action.
Role: Predicts the new state resulting from taking an action (e.g., taking "Up" increments y coordinate by 1).

2. Sensor Model (How sensory inputs reflect world state & update internal memory)

Code Implementation: Adds the current position to self.visited_cells and uses percept["wall_ahead"] to prune invalid candidate moves (e.g., removing "Up" if blocked by a wall).
Role: Integrates current sensory input with the world model to ensure the state representation remains accurate.

3. Overall Impact

Before selecting an action, the agent queries its updated internal state memory:
unvisited_moves = [move for move, pos in candidates.items() if pos not in self.visited_cells]
By prioritizing unexplored positions, the agent breaks out of reflex loops and successfully navigates partially observable environments.

Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.